

std::constexpr_v

Document #: P2781R2
Date: 2023-05-17
Project: Programming Language C++
Audience: LEWG-I
LEWG
Reply-to: Matthias Kretz
<m.kretz@gsi.de>
Zach Laine
<whatwasthataddress@gmail.com>

Contents

- 1 Changelog
 - 1.1 Changes since R0
 - 1.2 Changes since R1
- 2 Relationship to previous work
- 3 The ergonomics of std::integral_constant<int> are bad
 - 3.1 Replacing the uses of std::integral_constant is not enough
 - 3.2 The difference in template parameters to std::constexpr_v and std::c_
- 4 Making constexpr_v more useful
- 5 What about strings?
- 6 An example using operator()
- 7 What about the mutating operators?
 - 7.0.1 Possible LEWG poll
- 8 What about operator->?
- 9 Convertibility to and from std::integral_constant
- 10 Design
 - 10.1 Add constexpr_v
 - 10.2 Add a feature macro
- 11 Implementation experience
- 12 Possible polls for LEWG
- 13 Wording

§ 1 Changelog

§ 1.1 Changes since R0

- Remove discussion of literals for std::integral_constant and std::constexpr_v, based on LEWG feedback.
- Change the title to reflect the loss of the literals.

- Add an explicit example involving `std::constexpr_v::operator()`, as requested by LEWG.
- Add concept `std::constexpr_value`, as suggested by LEWG.
- Wording.

§ 1.2 Changes since R1

- Remove the `constexpr_value` concept.
- Use an auto NTTP parameter for `constexpr_v`, and default its type template parameter.
- Add the option for adding the mutating overloadable operators.
- Add a note about `operator->`.
- Add remarks about interconvertibility with `std::integral_constant`.
- Reduce the number of naming options.
- Simplify the implementation.

§ 2 Relationship to previous work

This paper is co-authored by the authors of P2725R1 (“`std::integral_constant` Literals”) and P2772R0 (“`std::integral_constant` literals do not suffice — `constexpr_t`?”). This paper supersedes both of those previous papers.

§ 3 The ergonomics of `std::integral_constant<int>` are bad

`std::integral_constant<int>` is used in lots of places to communicate a constant integral value to a given interface. The length of its spelling makes it very verbose. Fortunately, we can do a lot better.

Before	After
<pre>// From P2630R1 auto const sir = std::strided_index_range{std::integral_constant<size_t, 0>{}, std::integral_constant<size_t, 10> {}, 3}; auto y = submdspan(x, sir);</pre>	<pre>auto y = submdspan(x, std::strided_index_range{ std::c_<0>, std::c_<10>, 3});</pre>

The “after” case above would require that `std::strided_index_range` be changed; that is not being proposed here. The point of the example is to show the relative convenience of `std::integral_constant` versus the proposed `std::constexpr_v`.

§ 3.1 Replacing the uses of `std::integral_constant` is not enough

Parameters passed to a `constexpr` function lose their `constexpr`-ness when used inside the function. Replacing `std::integral_constant` with `std::constexpr_v` has the potential to improve a lot more uses of compile-time constants than just integrals; what about all the other `constexpr`-friendly C++ types?

Consider:

```
template<typename T>
struct my_complex
{
    T re, im;
```

```
};

inline constexpr short foo = 2;

template<typename T>
struct X
{
    void f(auto c)
    {
        // c is to be used as a constexpr value here
    }
};
```

We would like to be able to call `x.f()` with a value, and have that value keep its constexpr-ness. Let's introduce a template "constexpr_v" that holds a constexpr value that it is given as a non-type template parameter.

```
namespace std {
    template<auto X, class T/* = remove_cvref_t<decltype(X)>*/>
    struct constexpr_v
    {
        using value_type = T;
        using type = constexpr_v;

        constexpr operator value_type() const { return X; }
        static constexpr value_type value = X;

        // The rest of the members are discussed below ....
    };
}
```

Now we can write this.

```
template<typename T>
void g(X<T> x)
{
    x.f(std::constexpr_v<1>{});
    x.f(std::constexpr_v<2uz>{});
    x.f(std::constexpr_v<3.0>{});
    x.f(std::constexpr_v<4.f>{});
    x.f(std::constexpr_v<foo>{});
    x.f(std::constexpr_v<my_complex(1.f, 1.f)>{});
}
```

Let's now add a constexpr variable template with a shorter name, say `c_`.

```
namespace std {
    template<auto X>
    inline constexpr constexpr_v<X> c_{};
}
```

And now we can write this.

```
template<typename T>
void g(X<T> x)
{
    x.f(std::c_<1>);
    x.f(std::c_<2uz>);
    x.f(std::c_<3.0>);
    x.f(std::c_<4.f>);
    x.f(std::c_<foo>);
    x.f(std::c_<my_complex(1.f, 1.f)>);
}
```

§ 3.2 The difference in template parameters to `std::constexpr_v` and `std::c_`

`std::c_` takes an auto NTTP. `std::constexpr_v` takes an auto NTTP `x`, and a type `T` which is defaulted to `decltype(x)`. Why is this? ADL! Even though the type of `x` is deduced with or without `T`, without `T` some natural uses of `constexpr_v` cease to work. For instance:

```
auto f = std::c_<strlit("foo")>; // Using the strlit from later in this paper.
std::cout << f << "\n";
```

The stream insertion breaks without the `T` parameter. The `T` parameter is `strlit</*...*/>`, which pulls `strlit`'s `operator<<` into consideration during ADL.

§ 4 Making `constexpr_v` more useful

`constexpr_v` is essentially a wrapper. It takes a value `x` of some structural type `T`, and represents `x` in such a way that we can continue to use `x` as a compile-time constant, regardless of context. As such, `constexpr_v` should be implicitly convertible to `T`; this is already reflected in the design presented above. For the same reason, `constexpr_v` should provide all the operations that the underlying type has. Though we cannot predict what named members the underlying type `T` has, we *can* guess at all the operator overloads it might have.

So, by adding conditionally-defined overloads for all the overloadable operators, we can make `constexpr_v` as natural to use as many of the types it might wrap.

```
namespace std {
    template<auto X, class T/* = remove_cvref_t<decltype(X)>*/>
    struct constexpr_v {
        using value_type = T;
        using type = constexpr_v;

        constexpr operator value_type() const { return X; }
        static constexpr value_type value = X;

        // unary -
        template<auto Y = X>
        constexpr auto operator-() const -> constexpr_v<-Y> { return {}; }

        // binary + and -
        template<lhs-constexpr-param<type> U, constexpr-param V>
        friend constexpr constexpr_v<U::value + V::value> operator+(U, V) { return {}; }
        template<lhs-constexpr-param<type> U, constexpr-param V>
        friend constexpr constexpr_v<U::value - V::value> operator-(U, V) { return {}; }

        // etc... (full listing later)
    };
}
```

These operators are defined in such a way that they behave just like the operations on underlying the `T` and `v` values would, including promotions and coercions. For example:

```
static_assert(std::is_same_v<
    decltype(std::c_<42> - std::c_<13u>),
    std::constexpr_v<29u>>);
```

Each operation is only defined if the underlying operation on `x` and `y` is defined. Each operation additionally requires that the result of the underlying operation have a structural type.

The mutating operations are left out, because none of them makes sense – the type of the mutated value would have to change, since the value is itself part of the type.

All the remaining operations are included, even the index and call operators. The rationale for this is that a user may want to make some sort of compile-time domain-specific embedded language using operator overloading, and having all but a couple of the operators specified would frustrate that effort.

The only downside to adding `std::constexpr_v::operator()()` is that it would represent a break from the design of `std::integral_constant`, making it an imperfect drop-in replacement for that template.

The operators are designed to interoperate with other types and templates that have a `constexpr` static value member. This works with `std::constexpr_vs` of course, but also `std::integral_constants`, and user-provided types as well. For example:

```
struct my_type { constexpr static int value = 42; };

void foo()
{
    constexpr auto zero = my_type{} - std::c_<42>; // Ok.
    // ...
}
```

Note that the addition of these operators is in line with the poll:

“Add a new robust integral constant type with all the numerical operators, as proposed in P2772R0, and use that for these literals instead of `std::integral_constant`”?

SF	F	N	A	SA
4	7	1	1	1

... taken in the 2023-01-17 Library Evolution telecon.

Note that the one SA said he would not be opposed if the word “integral” was stricken from the poll, and the design of `std::constexpr_v` is not limited to integral types.

§ 5 What about strings?

As pointed out on the reflector, `std::c_<"foo">` does not work, because of language rules. However, it's pretty easy for users to add an NTTP-friendly string wrapper type, and then use that with `std::c_<>`.

```
template<size_t N>
struct strlit
{
    constexpr strlit(char const (&str)[N]) { std::copy_n(str, N, value); }

    template<size_t M>
    constexpr bool operator==(strlit<M> rhs) const
    {
        return std::ranges::equal(bytes_, rhs.bytes_);
    }

    friend std::ostream & operator<<(std::ostream & os, strlit l)
    {
        assert(!l.value[N - 1] && "value must be null-terminated");
        return os.write(l.value, N - 1);
    }

    char value[N];
};

int main()
{
    auto f = std::c_<strlit("foo")>;
    std::cout << f; // Prints "foo".
}
```

§ 6 An example using operator()

The addition of non-arithmetic operators may seem academic at first. However, consider this constexpr-friendly parser combinator mini-library.

```
namespace parse {

    template<typename L, typename R>
    struct or_parser;

    template<size_t N>
    struct str_parser
    {
        template<size_t M>
        constexpr bool operator()(strlit<M> lit) const
        {
            return lit == str_;
        }
        template<typename P>
        constexpr auto operator|(P parser) const
        {
            return or_parser<str_parser, P>{*this, parser};
        }
        strlit<N> str_;
    };

    template<typename L, typename R>
    struct or_parser
    {
        template<size_t M>
        constexpr bool operator()(strlit<M> lit) const
        {
            return l_(lit) || r_(lit);
        }
        template<typename P>
        constexpr auto operator|(P parser) const
        {
            return or_parser<or_parser, P>{*this, parser};
        }
        L l_;
        R r_;
    };
}

int foo()
{
    constexpr parse::str_parser p1{strlit("neg")};
    constexpr parse::str_parser p2{strlit("incr")};
    constexpr parse::str_parser p3{strlit("decr")};

    constexpr auto p = p1 | p2 | p3;

    constexpr bool matches_empty = p(strlit(""));
    static_assert(!matches_empty);
    constexpr bool matches_pos = p(strlit("pos"));
    static_assert(!matches_pos);
    constexpr bool matches_decr = p(strlit("decr"));
    static_assert(matches_decr);
}
```

(This relies on the `strlit` struct shown just previously.)

Say we wanted to use the templates in namespace `parser` along side other values, like `ints` and `floats`. We would want that not to break our `std::constexpr_v` expressions. Having to work around the absence of `std::constexpr_v::operator()` would require us to write a lot more code. Here is the equivalent of the function `foo()` above, but with all the variables wrapped using `std::c_`.

```
int bar()
{
    constexpr parse::str_parser p1{strlit("neg")};
    constexpr parse::str_parser p2{strlit("incr")};
    constexpr parse::str_parser p3{strlit("decr")};

    constexpr auto p_ = std::c_<p1> | std::c_<p2> | std::c_<p3>;

    constexpr bool matches_empty_ = p_(std::c_<strlit("")>);
    static_assert(!matches_empty_);
    constexpr bool matches_pos_ = p_(std::c_<strlit("pos")>);
    static_assert(!matches_pos_);
    constexpr bool matches_decr_ = p_(std::c_<strlit("decr")>);
    static_assert(matches_decr_);
}
```

As you can see, everything works as it did before. The presence of `operator()` does not enable any new functionality, it just keeps code that happens to use it from breaking.

§ 7 What about the mutating operators?

The operators left out of the code below are the ones that involve mutation, like `operator++`, `operator/=`, etc. These seem at first that these are nonsensical, since all the operations on a `constexpr_v` must be nonmutating.

However, some DSLs may wish to use these operations with atypical semantics.

```
struct weirdo
{
    constexpr int operator++() const { return 1; }
};
auto result = ++std::c_<weirdo{}>;
```

`result` is obviously `std::c_<1>` here, and no mutation occurred. You can imagine a more elaborate use case, say a library that is used to create expression templates. For example:

```
auto expr = std::c_<var0> += std::c_<var1>;
```

In this case, `var0` and `var1` would be some terminal types in the expression template library, and `operator+=` would return a `constexpr` expression tree, rather than mutating the left side of the `+=`.

Since this is a relatively late addition – after the paper has been through two LEWG reviews, the addition of these operators is being presented as an option. However, we have implemented it, and know that they work.

The optional parts in the design listing and wording are marked with `#if LEWG_SAYS_SO`.

§ 7.0.1 Possible LEWG poll

We want to add all overloadable operators to `constexpr_v`, including the ones that are usually mutating.

§ 8 What about operator ->?

We're not proposing it, because of its very specific semantics – it must yield a pointer, or something that eventually does. That's not a very useful operation during constant evaluation.

§ 9 Convertibility to and from `std::integral_constant`

During the LEWG reviews, some attendees suggested that inter-conversions between `std::integral_constant` and `std::constexpr_v` would be useful. The important thing to remember is that we want deduction to occur

when calling functions that take a `std::constexpr_v`, including the `std::constexpr_v` operator overloads. Conversions and deductions are at odds with one another, because deducing parameter types disables the conversion rules.

If you look at the operator overloads proposed here, you will see that they are deduction operations at their most essential. The types of the parameters do not matter, except that each conveys a value that is a core constant expression because it is embedded in the type system. The fact that a `std::constexpr_v` conveys that value instead of a `std::integral_constant` is immaterial, and in fact the operators are written in such a way that they operate on either template (as long as at least one parameter is a specialization of `std::constexpr_v`). Users can and should write their code using these kinds of values-as-types in a similar way. Relying on conversions is a less-useful way to get interoperability.

§ 10 Design

§ 10.1 Add `constexpr_v`

```
namespace std {
    template<auto X, class T = remove_cvref_t<decltype(X)>>
        struct constexpr_v;

    template <class T>
        concept constexpr-param = // exposition only
            requires { typename constexpr_v<T::value>; } and not
            is_member_pointer_v<decltype(&T::value)>;
    template <class T>
        concept derived-from-constexpr = // exposition only
            derived_from<T, constexpr_v<T::value>>;
    template <class T, class SelfT>
        concept lhs-constexpr-param = // exposition only
            constexpr-param<T> and (derived_from<T, SelfT> or not derived-from-constexpr<T>);

    template<auto X, class T>
    struct constexpr_v {
        using value_type = T;
        using type = constexpr_v;

        constexpr operator value_type() const { return X; }
        static constexpr value_type value = X;
    };

#ifdef LEWG_SAYS_S0
    template <constexpr-param U>
        constexpr constexpr_v<X = U::value> operator=(U) const { return {}; }
#endif

    template<auto Y = X>
        constexpr auto operator+() const -> constexpr_v<+Y> { return {}; }
    template<auto Y = X>
        constexpr auto operator-() const -> constexpr_v<-Y> { return {}; }
    template<auto Y = X>
        constexpr auto operator~() const -> constexpr_v<~Y> { return {}; }
    template<auto Y = X>
        constexpr auto operator!() const -> constexpr_v<!Y> { return {}; }
    template<auto Y = X>
        constexpr auto operator&() const -> constexpr_v<&Y> { return {}; }
    template<auto Y = X>
        constexpr auto operator*() const -> constexpr_v<*Y> { return {}; }

    template<class... Args>
        constexpr auto operator()(Args... args) const -> constexpr_v<X(Args::value...)> { return
            {}; }
    template<class... Args>
        constexpr auto operator[](Args... args) const -> constexpr_v<X[Args::value...]> { return
            {}; }
```


[illegible]

```

        friend constexpr constexpr_v<(U::value >= V::value)> operator>=(U, V) { return {}; }
    #endif
};

template<auto X>
    inline constexpr constexpr_v<X> c_{};
}

```

§ 10.2 Add a feature macro

Add a new feature macro, `__cpp_lib_constexpr_v`.

§ 11 Implementation experience

Look up a few lines to see an implementation of `std::constexpr_v`. At the time of this writing, there is one caveat: `operator[]()` looks correct to the authors, but does not work in any compiler tested, due to the very limited multi-variate `operator[]` support in even the latest compilers.

Additionally, an `integral_constant` with most of the operator overloads has been a part of [Boost.Hana](#) since its initial release in May of 2016. Its operations have been used by many, many users.

§ 12 Possible polls for LEWG

— We should call `std::constexpr_v`:

1. `std::constexpr_wrapper`

- Pro: The name calls out that this type has a relation to constant expressions.
- Pro: Consistency. The name is analogous to `std::reference_wrapper` (and `std::ref`).
- Pro: `constexpr_wrapper<1>` reads as **wrapper** for **constant expression** with value **1**.
- Con: It's a long name. However, if we expect that user typically won't spell out this type, then it doesn't matter much.

2. `std::constexpr_t`

- Pro: The name calls out that this type has a relation to constant expressions.
- Pro: Read the name as either “the result of a **constant expression** identified by a **type**” or “a **type** identifying a value usable in **constant expressions**”.
- Pro: The name calls out that it's a type (or class template) and not a variable template.
- Con: `_t` in the standard library typically means “alias template for `::type` member of a trait”. The use of `_t` to identify types is more of a C thing.

3. `std::constexpr_value`

- Pro: The name calls out that this type has a relation to constant expressions.
- Pro: `constexpr_value<1>` can be read as “the value 1 as a constant expression”.
- Con: The `_value` suffix may mislead readers to expect a variable template.

4. `std::constant_value`

- Con: The `_value` suffix may mislead readers to expect a variable template.
- Con: Nothing in the type hints at the primary use case: enabling *constant expressions*.

- Con: The name constant hints at const. While const isn't wrong here, it's also irrelevant.
- Con: `constant_value<1>` reads pretty much like `const int` with value 1, which is misleading.
- We should call `std::c_`:
 1. `std::cc`
 - short for `constexpr_wrapper` constant (somewhat silly, sure)
 - quick to type: hit the same key twice
 - 203'614 matches found on codesearch.isocpp.org
 2. `std::c_`
 - hardest to type (IMHO and with US keyboard layout, i.e. type “c shift+_ shift+<”; the _ < movement is slowing me down)
 - 10'198 matches found on codesearch.isocpp.org
 3. `std::cw`
 - short for `constexpr_wrapper` or `constexpr wrap`, i.e. read `cw<1>` as “constexpr wrap 1”
 - 41'311 matches found on codesearch.isocpp.org
 4. `std::c`
 - 3'869'416 matches found on codesearch.isocpp.org
 - scarily short

§ 13 Wording

Add the following to `[meta.type.synop]`, after `false_type`:

```
template<auto X, class T = remove_cvref_t<decltype(X)>>
  struct constexpr_v;

template <class T>
  concept constexpr-param =
    requires { typename constexpr_v<T::value>; } and not
    is_member_pointer_v<decltype(&T::value)>;
template <class T>
  concept derived-from-constexpr = // exposition only
    derived_from<T, constexpr_v<T::value>>;
template <class T, class SelfT>
  concept lhs-constexpr-param = // exposition only
    constexpr-param<T> and (derived_from<T, SelfT> or not derived-from-constexpr<T>);

template<auto X>
  inline constexpr constexpr_v<X> c_;
```

Add the following to `[meta.help]`, after `integral_constant`:

```
template<auto X, class T>
struct constexpr_v {
  using value_type = T;
  using type = constexpr_v;

  constexpr operator value_type() const { return X; }
  static constexpr value_type value = X;
};

#if LEWG_SAYS_SO
```

```

template <constexpr-param U>
    constexpr constexpr_v<X = U::value> operator=(U) const { return {}; }
#endif

template<auto Y = X>
    constexpr auto operator+() const -> constexpr_v<+Y> { return {}; }
template<auto Y = X>
    constexpr auto operator-() const -> constexpr_v<-Y> { return {}; }
template<auto Y = X>
    constexpr auto operator~() const -> constexpr_v<~Y> { return {}; }
template<auto Y = X>
    constexpr auto operator!() const -> constexpr_v<!Y> { return {}; }
template<auto Y = X>
    constexpr auto operator&() const -> constexpr_v<&Y> { return {}; }
template<auto Y = X>
    constexpr auto operator*() const -> constexpr_v<*Y> { return {}; }

template<class... Args>
    constexpr auto operator()(Args... args) const -> constexpr_v<X(Args::value...)> { return
        {};}
template<class... Args>
    constexpr auto operator[](Args... args) const -> constexpr_v<X[Args::value...]> { return
        {};}

template <lhs-constexpr-param<type> U, constexpr-param V>
    friend constexpr constexpr_v<U::value + V::value> operator+(U, V) { return {};}
template <lhs-constexpr-param<type> U, constexpr-param V>
    friend constexpr constexpr_v<U::value - V::value> operator-(U, V) { return {};}
template <lhs-constexpr-param<type> U, constexpr-param V>
    friend constexpr constexpr_v<U::value * V::value> operator*(U, V) { return {};}
template <lhs-constexpr-param<type> U, constexpr-param V>
    friend constexpr constexpr_v<U::value / V::value> operator/(U, V) { return {};}
template <lhs-constexpr-param<type> U, constexpr-param V>
    friend constexpr constexpr_v<U::value % V::value> operator%(U, V) { return {};}

template <lhs-constexpr-param<type> U, constexpr-param V>
    friend constexpr constexpr_v<(U::value << V::value)> operator<<(U, V) { return {};}
template <lhs-constexpr-param<type> U, constexpr-param V>
    friend constexpr constexpr_v<(U::value >> V::value)> operator>>(U, V) { return {};}
template <lhs-constexpr-param<type> U, constexpr-param V>
    friend constexpr constexpr_v<U::value & V::value> operator&(U, V) { return {};}
template <lhs-constexpr-param<type> U, constexpr-param V>
    friend constexpr constexpr_v<U::value | V::value> operator|(U, V) { return {};}
template <lhs-constexpr-param<type> U, constexpr-param V>
    friend constexpr constexpr_v<U::value ^ V::value> operator^(U, V) { return {};}

template <lhs-constexpr-param<type> U, constexpr-param V>
    friend constexpr constexpr_v<U::value && V::value> operator&&(U, V) { return {};}
template <lhs-constexpr-param<type> U, constexpr-param V>
    friend constexpr constexpr_v<U::value || V::value> operator||(U, V) { return {};}

template <lhs-constexpr-param<type> U, constexpr-param V>
    friend constexpr constexpr_v<(U::value <=> V::value)> operator<=>(U, V) { return {};}
template <lhs-constexpr-param<type> U, constexpr-param V>
    friend constexpr constexpr_v<(U::value == V::value)> operator==(U, V) { return {};}
template <lhs-constexpr-param<type> U, constexpr-param V>
    friend constexpr constexpr_v<(U::value != V::value)> operator!=(U, V) { return {};}
template <lhs-constexpr-param<type> U, constexpr-param V>
    friend constexpr constexpr_v<(U::value < V::value)> operator<(U, V) { return {};}
template <lhs-constexpr-param<type> U, constexpr-param V>
    friend constexpr constexpr_v<(U::value > V::value)> operator>(U, V) { return {};}
template <lhs-constexpr-param<type> U, constexpr-param V>
    friend constexpr constexpr_v<(U::value <= V::value)> operator<=(U, V) { return {};}
template <lhs-constexpr-param<type> U, constexpr-param V>
    friend constexpr constexpr_v<(U::value >= V::value)> operator>=(U, V) { return {};}

template <lhs-constexpr-param<type> U, constexpr-param V>

```

```

    friend constexpr constexpr_v<(U::value, V::value)> operator,(U, V) { return {}; }
template <lhs-constexpr-param<type> U, constexpr-param V>
    friend constexpr constexpr_v<(U::value ->* V::value)> operator->*(U, V) { return {}; }

#if LEWG_SAYS_SO
template <lhs-constexpr-param<type> U, constexpr-param V>
    friend constexpr constexpr_v<(U::value += V::value)> operator+=(U, V) { return {}; }
template <lhs-constexpr-param<type> U, constexpr-param V>
    friend constexpr constexpr_v<(U::value -= V::value)> operator-=(U, V) { return {}; }
template <lhs-constexpr-param<type> U, constexpr-param V>
    friend constexpr constexpr_v<(U::value *= V::value)> operator*=(U, V) { return {}; }
template <lhs-constexpr-param<type> U, constexpr-param V>
    friend constexpr constexpr_v<(U::value /= V::value)> operator/=(U, V) { return {}; }
template <lhs-constexpr-param<type> U, constexpr-param V>
    friend constexpr constexpr_v<(U::value %= V::value)> operator%=(U, V) { return {}; }
template <lhs-constexpr-param<type> U, constexpr-param V>
    friend constexpr constexpr_v<(U::value &= V::value)> operator&=(U, V) { return {}; }
template <lhs-constexpr-param<type> U, constexpr-param V>
    friend constexpr constexpr_v<(U::value |= V::value)> operator|=(U, V) { return {}; }
template <lhs-constexpr-param<type> U, constexpr-param V>
    friend constexpr constexpr_v<(U::value ^= V::value)> operator^=(U, V) { return {}; }
template <lhs-constexpr-param<type> U, constexpr-param V>
    friend constexpr constexpr_v<(U::value <= V::value)> operator<=(U, V) { return {}; }
template <lhs-constexpr-param<type> U, constexpr-param V>
    friend constexpr constexpr_v<(U::value >= V::value)> operator>=(U, V) { return {}; }
#endif
};

template<auto X>
    inline constexpr constexpr_v<X> c_{};

```

- 2 The class template `constexpr_v` aids in metaprogramming by ensuring that the evaluation of expressions comprised entirely of `constexpr_vs` are core constant expressions ([`expr.const`]), regardless of the context in which they appear. In particular, this enables use of `constexpr_v` values that are passed as arguments to `constexpr` functions to be used as template parameters.
- 3 The variable template `c_` is provided as a convenient way to nominate `constexpr_v` values.

Add to [version.syn]:

```
#define __cpp_lib_constexpr_v XXXXXL // also in <type_traits>
```